```c
#include <stdio.h>

int add(int a, int b);
float average(int a, int b, int c);

int main()
{
    int age = "Twenty";
    // ERROR: Cannot assign a string literal to an int variable.
    // CORRECTION: int age = 20;
    // OR: char age[] = "Twenty";

    float salary = 50000.50;

    int marks = 85;

    char grade = "A";
    // ERROR: Double quotes create a string literal, not a character.
    // CORRECTION: char grade = 'A';

    printf("%d\n", salary);
    // ERROR: %d is for int, but salary is float.
    // CORRECTION: printf("%f\n", salary);

    printf("%f\n", marks);
    // ERROR: %f is for float, but marks is int.
    // CORRECTION: printf("%d\n", marks);

    int total;

    total = marks + score;
    // ERROR: Variable 'score' is undeclared.
    // CORRECTION:
    // int score = 10;
    // total = marks + score;

    int x = 10;
    int x = 20;
    // ERROR: Redeclaration of variable 'x' in the same scope.
    // CORRECTION: x = 20;
    // OR: int y = 20;
```

```
26.
27.     100 = x;
        // ERROR: Cannot assign a value to a constant.
        // CORRECTION: x = 100;
28.
29.     int num = 25;
30.
31.     printf("%s\n", num);
        // ERROR: %s expects a string, but num is int.
         // CORRECTION: printf("%d\n", num);
32.
33 .    float pi = 3.14;
34.
35.     printf("%d\n", pi);
        // ERROR: %d is for int, but pi is float.
        // CORRECTION: printf("%f\n", pi);
36.
37.     int arr[5];
38.
39.     arr[2.5] = 100;
        // ERROR: Array index must be an integer.
        // CORRECTION: arr[2] = 100;
40.
41.     char ch = 65.5;
        // WARNING: Implicit conversion from float to char causes data loss.
        // CORRECTION: char ch = 65;
        // OR: float ch = 65.5;
42.
43.     int result;
44.
45.     result = add(10);
        // ERROR: Function add() requires two arguments.
        // CORRECTION: result = add(10, 20);
46.
47.     printf("%d\n", result);
48.
49.     float avg;
50.
51.     avg = average(10,20);
        // ERROR: Function average() requires three arguments.
        // CORRECTION: avg = average(10, 20, 30);
```

```
52.
53.    printf("%f\n", avg);
54.
55.    int temperature = "35";
       // ERROR: Cannot assign a string literal to an int.
       // CORRECTION: int temperature = 35;
56.
57.    int amount = 1000;
58.
59.    float discount = "10.5";
       // ERROR: Cannot assign a string literal to a float.
       // CORRECTION: float discount = 10.5;
60.
61.    amount = amount - discount;
       // WARNING: Assigning float result to int may lose precision.
       // CORRECTION: amount = amount - (int)discount;
       // OR: change amount to float.
62.
63.    char name[20];
64.
65.    name = "Robil";
       // ERROR: Arrays cannot be assigned after declaration.
       // CORRECTION:
       // strcpy(name, "Robil");   // Requires #include <string.h>
       // OR: char name[20] = "Robil";
66.
67.    printf("%s\n", name);
68.
69.    int count;
70.
71.    count = totalMarks + 10;
       // ERROR: Variable 'totalMarks' is undeclared.
       // CORRECTION:
       // int totalMarks = 100;
       // count = totalMarks + 10;
72.
73.    int number = 50;
74.
75.    printf("%c\n", number);
       // WARNING: %c prints the ASCII character for value 50.
       // CORRECTION: printf("%d\n", number);
```

```

float rate = 7.5;

printf("%d\n", rate);
// ERROR: %d is for int, but rate is float.
// CORRECTION: printf("%f\n", rate);

int a = 10;
int b = 20;

int sum;

sum = add(a,b,c);
// ERROR: Variable 'c' is undeclared.
// ERROR: Function add() accepts only two arguments.
// CORRECTION: sum = add(a, b);

printf("%d\n", sum);

float radius = 5.5;

int area;

area = 3.14 * radius * radius;
// WARNING: Float result assigned to int causes loss of precision.
// CORRECTION:
// float area;
// area = 3.14f * radius * radius;

printf("%d\n", area);
// ERROR: If area is changed to float, %d becomes invalid.
// CORRECTION: printf("%f\n", area);

int studentMarks = 90;

if(studentMarks = 100)
// ERROR: Assignment operator (=) used instead of comparison operator (==).
// CORRECTION: if(studentMarks == 100)
{
  printf("Perfect Score\n");
}
```

```
int size = "Large";
// ERROR: Cannot assign a string literal to an int.
// CORRECTION: char size[] = "Large";

printf("%d\n", size);
// ERROR: If size is a string, %d is invalid.
// CORRECTION: printf("%s\n", size);

float percentage = 89.75;

printf("%d\n", percentage);
// ERROR: %d is for int, but percentage is float.
// CORRECTION: printf("%f\n", percentage);

int arr2[10];

arr2["five"] = 100;
// ERROR: Array index must be an integer.
// CORRECTION: arr2[5] = 100;

int choice = 2;

switch(choice)
{
    case 1:
        printf("One\n");
        break;

    case "2":
// ERROR: Case label must be an integer constant.
// CORRECTION: case 2:

        printf("Two\n");
        break;
}

int finalResult;

finalResult = multiply(10,20);
// ERROR: Function 'multiply()' is not declared or defined.
```

```
// CORRECTION:
// int multiply(int a, int b);
// int multiply(int a, int b) { return a * b; }

printf("%d\n", finalResult);

int length = 10;
int width = 5;

float areaRect;

areaRect = length * width;

printf("%d\n", areaRect);
// ERROR: %d is for int, but areaRect is float.
// CORRECTION: printf("%f\n", areaRect);

int marks1 = 70;
int marks2 = 80;
int marks3 = 90;

int avgMarks;

avgMarks = average(marks1, marks2, marks3);
// WARNING: average() returns float, but avgMarks is int.
// CORRECTION:
// float avgMarks;
// avgMarks = average(marks1, marks2, marks3);

printf("%d\n", avgMarks);
// ERROR: If avgMarks is changed to float, %d is invalid.
// CORRECTION: printf("%f\n", avgMarks);

int data;

printf("%d\n", value);
// ERROR: Variable 'value' is undeclared.
// CORRECTION:
// int value = 0;
// printf("%d\n", value);

```

```
 int employeeId = "EMP101";
// ERROR: Cannot assign a string literal to an int.
// CORRECTION: char employeeId[] = "EMP101";

 printf("%d\n", employeeId);
// ERROR: %d is for int, but employeeId is a string.
// CORRECTION: printf("%s\n", employeeId);

 return 0;
}

int add(int a, int b)
{
        return a + b;
}

float average(int a, int b, int c)
{
        return (a + b + c) / 3.0;
}
```